

Trabajo Práctico: Descubriendo las redes

Cohorte de Marzo del 2026

Alumnos

Ryduan Quinteros Camaño

Valentina Fernandez Basualdo

Tecnicatura Universitaria en Programación - Universidad Tecnológica Nacional.

Programación I

Docente Titular

Alberto Cortez

Docente Tutor

Flor Camila Gubiotti

17 de Junio de 2026

Marco Teórico del Proyecto	1
Introducción	1
Gestión, Persistencia y Estructura de Datos	1
Modularización y Lógica de Control	1
Algoritmia y Procesamiento de Datos	1
Bibliografía y Referencias	2
Listas	2
Diccionarios	3
Funciones y Modularización	3
Estructuras Condicionales	4
Estructuras Repetitivas	4
Archivos CSV	4
Manejo de Archivos y Persistencia de Datos	5
Validación de Datos y Manejo de Errores	5
Algoritmos de Búsqueda	6
Ordenamientos	6
Estadísticas Básicas	6
Conclusión del Marco Teórico	7
LINKS	11

Marco Teórico del Proyecto

Introducción

El presente proyecto se fundamenta en el desarrollo de un sistema de gestión y análisis de datos en Python, estructurado bajo un paradigma de programación modular. El sistema se divide en dos grandes ejes: el primero, orientado a la persistencia y gestión integral de la información (ABM), y el segundo, enfocado en el procesamiento algorítmico, filtrado y análisis estadístico de dicha información.

Gestión, Persistencia y Estructura de Datos

Para garantizar la integridad y persistencia de los datos, se ha implementado un sistema de gestión basado en archivos en formato CSV (Comma-Separated Values).

- **Persistencia:** La lectura y escritura se gestionan mediante el gestor de contexto with `open(...)`, una práctica recomendada que garantiza la liberación de recursos del sistema al finalizar la operación (Python Software Foundation, 2026).
- **Estructuras de Datos:** Se utilizan listas de diccionarios. Esta elección permite representar entidades complejas (países) con atributos heterogéneos. Mientras que la lista actúa como un contenedor secuencial y mutable, el diccionario proporciona un acceso eficiente mediante claves, facilitando la escalabilidad del sistema (Lutz, 2013).
- **Programación Defensiva:** Con el objetivo de asegurar la integridad del sistema, se aplicaron validaciones en los puntos de entrada (input) mediante bucles de control (`while True`) y comprobaciones de tipo (`isdigit()`), evitando que el programa finalice inesperadamente ante datos mal formados.

Modularización y Lógica de Control

El sistema sigue el principio DRY (Don't Repeat Yourself), estructurando la lógica en bloques independientes (funciones).

- **Alcance de variables:** Se gestiona el scope priorizando variables locales sobre globales para evitar efectos secundarios.
- **Sentencia pass:** Se utiliza estratégicamente como marcador de posición (placeholder) durante la etapa de desarrollo para definir la estructura sintáctica de funciones pendientes de lógica.
- **Modularidad:** La división en módulos (gestión de archivos, interfaz, análisis) permite la independencia de las capas del programa, facilitando su mantenimiento y lectura.

Algoritmia y Procesamiento de Datos

La parte central del análisis de datos se apoya en algoritmos de procesamiento avanzado:

- **Algoritmos de Ordenamiento:** Se implementaron métodos para organizar la información bajo criterios específicos, tales como el Ordenamiento Burbuja mejorado (utilizando banderas de intercambio para optimizar el rendimiento al detectar listas ya ordenadas) y el Ordenamiento por Selección.

- Algoritmos de Búsqueda: Se utiliza la búsqueda lineal para coincidencias simples y la búsqueda binaria en casos donde la eficiencia es crítica, requiriendo previamente el ordenamiento del conjunto de datos.
- Análisis estadístico: Se emplean contadores y acumuladores para calcular promedios, máximos y mínimos mediante la estrategia de inicialización en infinito (`float('inf')`), garantizando que cualquier valor del conjunto reemplace al extremo registrado.
- List Comprehensions: Para los filtros de datos (por continente o métricas numéricas), se hace uso de esta técnica que permite crear subconjuntos de datos de forma declarativa y concisa, mejorando la legibilidad frente a la iteración tradicional (UTN, 2026).

Bibliografía y Referencias

- Cimino, C. (s.f.). Material de cátedra sobre ciclos, sumatorias y máximos/mínimos en Python.
- González, F. (s.f.). Composición de funciones y gestión de memoria (Paso por valor y referencia).
- Lutz, M. (2013). Learning Python. O'Reilly Media.
- Python Software Foundation. (2026). The Python Standard Library: csv — CSV File Reading and Writing. Recuperado de <https://docs.python.org/3/library/csv.html>
- The Regext. (s.f.). Algoritmos de ordenamiento y búsqueda en listas. [Archivo de Vídeo].
- UTN - Tecnicatura Universitaria en Programación a Distancia. (2026). Documentos de Cátedra: Listas, Arreglos estáticos, Funciones y Ámbito de variables.

Listas

Las listas son estructuras de datos que permiten almacenar múltiples elementos dentro de una misma variable. Son ordenadas, modificables y permiten agregar, eliminar o actualizar elementos durante la ejecución del programa.

En Python, las listas se representan mediante corchetes (`[]`) y pueden contener elementos de diferentes tipos de datos.

Ejemplo:

```
países = ["Argentina", "Brasil", "Chile"]
```

En el presente proyecto, las listas se utilizan para almacenar todos los países cargados desde el archivo CSV. Cada elemento de la lista representa un país y contiene toda la información asociada a dicho registro.

Las listas resultan fundamentales para realizar búsquedas, filtros, ordenamientos y cálculos estadísticos sobre el conjunto de datos.

Bibliografía

- Documentación Oficial de Python. Built-in Types – Lists.
- Material de Programación I – UTN.

Diccionarios

Los diccionarios son estructuras de datos que almacenan información mediante pares clave-valor. Cada clave permite acceder rápidamente al valor asociado.

Se representan utilizando llaves ({}).

Ejemplo:

```
pais = {  
    "nombre": "Argentina",  
    "poblacion": 45376763,  
    "superficie": 2780400,  
    "continente": "América"  
}
```

En este proyecto, cada país es representado mediante un diccionario. Esto permite organizar los datos de forma clara y acceder a ellos utilizando nombres descriptivos en lugar de posiciones numéricas.

El uso combinado de listas y diccionarios constituye la estructura principal de almacenamiento de información dentro del sistema.

Bibliografía

- Documentación Oficial de Python. Dictionary Objects.
- Material de Estructuras de Datos – UTN.

Funciones y Modularización

Las funciones son bloques de código diseñados para realizar una tarea específica. Permiten reutilizar instrucciones, evitar la duplicación de código y mejorar la organización del programa.

Ejemplo:

```
def buscar_pais():  
    pass
```

La modularización consiste en dividir un programa en funciones pequeñas e independientes, donde cada una posee una responsabilidad específica.

En este proyecto se implementan funciones para:

- Cargar datos desde el archivo CSV.
- Guardar información.
- Agregar países.
- Buscar registros.
- Actualizar datos.
- Filtrar información.
- Ordenar resultados.
- Calcular estadísticas.

Este enfoque facilita el mantenimiento del sistema y mejora su legibilidad.

Bibliografía

- Documentación Oficial de Python. Defining Functions.
- Material de Programación I – UTN.

Estructuras Condicionales

Las estructuras condicionales permiten que un programa tome decisiones dependiendo de determinadas condiciones.

En Python se implementan mediante las instrucciones:

- if
- elif
- else

Ejemplo:

```
if poblacion > 1000000:  
    print("País con gran población")  
else:  
    print("País con menor población")
```

Las estructuras condicionales son utilizadas en el sistema para validar datos ingresados por el usuario, verificar la existencia de registros y controlar distintas situaciones que pueden ocurrir durante la ejecución.

Bibliografía

- Python Software Foundation. Control Flow Tools.
- Material de Programación I – UTN.

Estructuras Repetitivas

Las estructuras repetitivas permiten ejecutar un conjunto de instrucciones múltiples veces. Python dispone principalmente de dos tipos de ciclos:

Ciclo for

Se utiliza para recorrer colecciones de datos.

```
for pais in paises:  
    print(pais["nombre"])
```

Ciclo while

Se utiliza cuando la cantidad de repeticiones depende de una condición.

```
while opcion != "4":  
    pass
```

En este proyecto los ciclos permiten recorrer listas de países, realizar búsquedas, validar entradas de usuario y procesar grandes cantidades de información de forma automática.

Bibliografía

- Documentación Oficial de Python. Loops and Iteration.
- Material de Programación I – UTN.

Archivos CSV

CSV (Comma Separated Values) es un formato de almacenamiento de datos basado en texto plano donde la información se organiza en filas y columnas.

Ejemplo:

```
nombre,poblacion,superficie,continente
```

Argentina,45376763,2780400,América
Brasil,213993437,8515767,América

Python incorpora el módulo csv, el cual permite leer y escribir este tipo de archivos de manera sencilla.

En el proyecto, el archivo CSV funciona como base de datos, almacenando toda la información de los países utilizados por el sistema.

Bibliografía

- Documentación Oficial de Python. csv Module.
- Material de Archivos CSV – UTN.

Manejo de Archivos y Persistencia de Datos

La persistencia de datos consiste en conservar información de forma permanente para que pueda recuperarse posteriormente.

Python permite trabajar con archivos mediante la función:

`open()`

y la estructura:

`with open(...)`

La utilización de `with` garantiza el cierre automático del archivo una vez finalizada la operación.

En este proyecto se implementa persistencia de datos mediante un archivo CSV que almacena los registros de países incluso después de cerrar el programa.

Bibliografía

- Documentación Oficial de Python. Reading and Writing Files.
- Material de Programación I – UTN.

Validación de Datos y Manejo de Errores

La validación de datos consiste en verificar que la información ingresada sea correcta antes de ser procesada.

Entre las validaciones implementadas en el sistema se encuentran:

- Control de campos vacíos.
- Verificación de datos numéricos.
- Validación de nombres.
- Control de duplicados.

Por otro lado, el manejo de errores permite detectar situaciones inesperadas sin provocar la finalización abrupta del programa.

Para ello Python utiliza bloques:

`try:`

`pass`

`except:`

`pass`

Estas estructuras permiten controlar errores durante la lectura del archivo CSV y mejorar la robustez del sistema.

Bibliografía

- Documentación Oficial de Python. Errors and Exceptions.
- Material de Manejo de Excepciones – UTN.

Algoritmos de Búsqueda

Los algoritmos de búsqueda permiten localizar información específica dentro de una colección de datos.

En este proyecto se implementa una búsqueda secuencial sobre la lista de países, recorriendo cada elemento hasta encontrar coincidencias con el criterio ingresado por el usuario.

Además de la búsqueda exacta, el sistema contempla búsquedas parciales para facilitar la localización de registros.

La búsqueda constituye una de las operaciones más importantes en cualquier sistema de gestión de información.

Bibliografía

- Material de Algoritmos de Búsqueda – UTN.
- Documentación Oficial de Python.

Ordenamientos

Los algoritmos de ordenamiento permiten reorganizar los datos siguiendo determinados criterios.

En el proyecto se implementan ordenamientos por:

- Nombre.
- Población.
- Superficie.

Los resultados pueden mostrarse en forma ascendente o descendente según la necesidad del usuario.

Los ordenamientos facilitan el análisis y la visualización de la información almacenada.

Bibliografía

- Documentación Oficial de Python. Sorting HOW TO.
- Material de Programación I – UTN.

Estadísticas Básicas

Las estadísticas básicas permiten obtener información resumida a partir de un conjunto de datos.

Entre los indicadores implementados en este proyecto se encuentran:

- País con mayor población.
- País con menor población.
- Promedio de población.
- Promedio de superficie.
- Cantidad de países por continente.

Estas métricas permiten transformar datos individuales en información útil para la toma de decisiones y el análisis de tendencias.

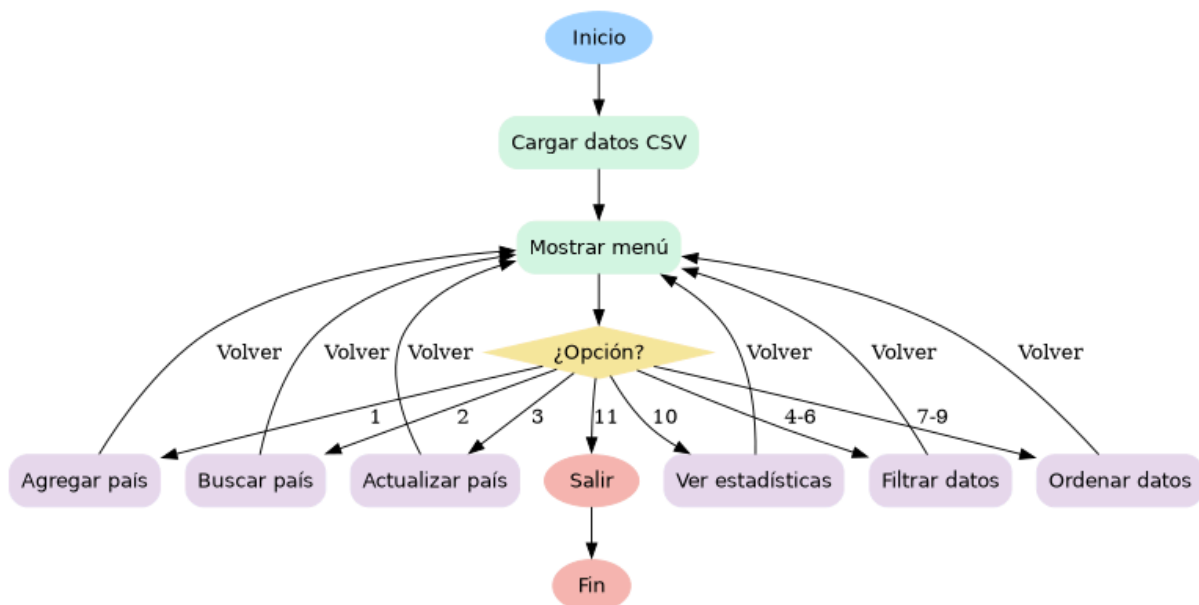
Bibliografía

- Material de Programación I – UTN.
- Downey, A. (2015). Think Python. O'Reilly Media.

Conclusión del Marco Teórico

Los conceptos desarrollados en este marco teórico constituyen la base para la construcción del sistema de gestión de países implementado en Python. La combinación de listas, diccionarios, funciones, archivos CSV, validaciones, búsquedas, ordenamientos y estadísticas permite desarrollar una aplicación capaz de almacenar, procesar y analizar información de manera eficiente, aplicando los conocimientos adquiridos durante la cursada de Programación I.

Diagrama



Capturas de pantalla

Opción 1

```
Seleccione una opción: 1

-- Agregar nuevo país --
Nombre: China
Población: 10000000
Superficie: 500000
Continente: Asiático
País agregado exitosamente.
```

Opción 2

Seleccione una opción: 2

Ingrese nombre del país a buscar: China

Resultados encontrados:

China | Población: 10000000 | Superficie: 500000 | Continente: Asiático

Opción 3

Seleccione una opción: 3

Ingrese el nombre del país a actualizar: China

Nueva población (actual: 10000000): 20000000

Nueva superficie (actual: 500000): 40000

Datos actualizados correctamente.

Opción 4

Seleccione una opción: 4

Ingrese continente: Asiático

Países encontrados:

China | Población: 20000000 | Superficie: 40000 | Continente: Asiático

Opción 5

Seleccione una opción: 5

Población mínima: 20000000

Población máxima: 40000000

Países encontrados:

China | Población: 20000000

Opción 6

```
Selecione una opción: 6

Superficie mínima: 20000
Superficie máxima: 100000

Países encontrados:

China | Superficie: 40000
```

Opción 7

```
Selecione una opción: 7

Países ordenados por nombre:

Argentina
China
```

Opción 8

```
Selecione una opción: 8
1. Ascendente
2. Descendente
Seleccione: 1

Países ordenados por población:

Argentina - 400000
China - 20000000
```

Opción 9

```
Selecione una opción: 9
1. Ascendente
2. Descendente
Seleccione: 1

Países ordenados por superficie:

China - 40000
Argentina - 500000
```

Opción 10

```
Seleccione una opción: 10

--- Estadísticas ---
Mayor población: China (20000000)
Menor población: Argentina (400000)
Promedio de población: 10200000.00
Promedio de superficie: 270000.00

Cantidad de países por continente:
Asiático: 1
America del Sur: 1
```

Opción 11

```
Seleccione una opción: 11
Sistema finalizado. ¡Hasta luego!
```

Reflexión grupal

El desarrollo de este TPI fue una experiencia clave para consolidar nuestra formación en Programación 1. Logramos integrar exitosamente la teoría con la práctica mediante un sistema robusto, aplicando modularización y manejo de excepciones para garantizar la integridad de los datos.

Este proyecto no solo nos permitió dominar el procesamiento de archivos y el uso de estructuras de datos (listas y diccionarios), sino que también reforzó nuestras habilidades de colaboración y gestión de proyectos. Al enfrentar desafíos reales de desarrollo y trabajar de manera coordinada en GitHub, logramos un producto final funcional, organizado y eficiente. En definitiva, este trabajo reafirma la importancia de un código limpio, bien documentado y pensado para la experiencia del usuario.

Dificultades y Soluciones Técnicas

Durante el desarrollo del proyecto, enfrentamos tres desafíos principales que fortalecieron nuestra capacidad técnica y de trabajo en equipo:

Gestión de control de versiones (GitHub): Tuvimos dificultades al sincronizar los cambios de código entre ambos integrantes, lo que provocó conflictos en las ramas. Lo solucionamos implementando un protocolo de comunicación antes de cada subida (push) y utilizando comandos de actualización (pull) constantes para asegurar que ambos trabajáramos siempre sobre la versión más reciente del código.

Integración de módulos: Al unir las partes desarrolladas por separado, surgieron inconsistencias en el flujo de datos. Para resolverlo, estandarizamos las estructuras de entrada y salida de cada función, asegurando que todas operaran de manera uniforme sobre la misma estructura de lista de diccionarios, y realizamos pruebas cruzadas para verificar la correcta integración.

Armado del menú interactivo: La creación del menú principal presentó el desafío de mantener el programa en ejecución sin cierres inesperados ante entradas erróneas. Lo

superamos aplicando un ciclo de validación (while True) acompañado de un bloque condicional (else), que captura cualquier opción no contemplada, permitiendo al usuario reintentar su elección de forma segura y profesional.

LINKS

GIT: <https://github.com/vale-m-j/Trabajo-Practico-Integrador-TPI-Programaci-n-1>

VIDEO: <https://youtu.be/K22zbIMVS1I>

Bibliografía de Referencia

Documentación y Fundamentos de Python:

- Python Software Foundation. (2026). Documentación oficial de Python 3.12.x. Recuperado de <https://docs.python.org/3/>
- Van Rossum, G., & Drake, F. L. (2020). The Python Tutorial. CreateSpace Independent Publishing Platform.

Material de Cátedra (UTN - Tecnicatura Universitaria en Programación):

- UTN - Tecnicatura Universitaria en Programación a Distancia. (s.f.). Materiales de estudio: Funciones, Arreglos estáticos, Listas, Matrices y Tipos de Datos Abstractos (TDA). [Documentos y actividades de la cátedra].

Material Audiovisual y Complementario:

- Cimino, C. (s.f.). Material audiovisual sobre estructuras de control, acumuladores, máximos y mínimos. [Videos instructivos de Programación 1].
- The Regext. (s.f.). Algoritmos de ordenamiento y búsqueda en Python. [Tutoriales audiovisuales].
- González, F. (s.f.). Material técnico sobre composición de funciones, alcance de variables y paso por valor/referencia. [Apuntes y transcripciones].